THE NATIONAL UNIVERSITY OF LESOTHO

Faculty of Science and Technology — Department of Mathematics and Computer Science

CS3520 — Computer Organisation and Architecture I

PRACTICAL LAB HANDOUT

Lab No.	Lab 3
Lab Title	Processor Simulation and Tracing — The Single-Cycle Datapath.
Supports Week(s)	Weeks 6–8 — the single-cycle datapath and its control.
Duration	Two 2-hour laboratory sessions.
Venue / Group(s)	ICT Lab — Group A (Wed 08:00–10:00) · Group B (Fri 08:00–10:00) · Group C (Fri 11:10–13:10)
Submission Platform	Numerical answers via the CS3520 Lab 3 Google Form (link on Thuto). Written work and code via your public GitHub repository, whose URL you submit on Thuto.
Submission Deadline	Before the start of the next lab session (see Thuto for the exact cut-off).

LEARNING OUTCOMES

On completion of this lab, you should be able to:

- Trace a RISC-V instruction through every element of a single-cycle datapath, and state the value carried on each port as it executes. (CLO3)
- Derive the control signals for each RISC-V instruction class from the observed behaviour of the hardware, and reconcile them with the Week 7 truth table. (CLO3)
- Compare a working implementation against the textbook datapath and justify, in engineering terms, why the two differ. (CLO3)

BEFORE YOU START

- Labs 0–2 completed. You should be able to write, assemble and step a short assembly program without help.
- The Week 6, 7 and 8 lectures. You will need the datapath diagram, the control truth table and the five-stage figure throughout. Bring them.
- Your fork of the course repository, updated. Part A will not start until you have the programs.
- A GitHub account, as in Lab 2. The written part of this lab is assessed from your own public repository.

This lab asks you to predict before you measure. Several steps say so explicitly. Write the prediction down first — a prediction recorded after the fact teaches you nothing, and the demonstrators will ask to see it.

TOOLS & RESOURCES

Tool / Resource	Purpose
Ripes	Processor simulation. Use the Extended layout throughout — the Standard layout hides the control unit and the signal wires, which are most of what this lab is about.
Course repository	github.com/khobatha/CS3520 — the five test programs used in this lab
Your own fork	Where your trace tables, written answers and your own programs are submitted from
CS3520 Lab 3 Google Form	Numerical answers. The link is posted on Thuto. One submission per student.
Week 6–8 lecture slides	The datapath figure, the control truth table and the five-stage pipeline diagram
RISC-V reference card	Opcode, funct3 and funct7 values for checking encodings by hand

PROCEDURE

PART A — THE SINGLE-CYCLE PROCESSOR

Weeks 6–7 made a claim about how a processor works. Here you check it against a working implementation, element by element.

Step 1 — Set Up the Simulator and Collect the Programs

Update your fork, then copy the lab programs into your own working folder:

```
git fetch upstream
git merge upstream/main
cp -r labs/2026/lab-3-processor-tracing/ ~/cs3520-lab3/
```

In Ripes, open the Select Processor dialog and choose:

- Processor: RISC-V Single Cycle Processor
- Layout: Extended

Then enable View → Show processor signal values. Every output port in the design will now display its current value, which is how you will read the datapath for the rest of Part A.

Four toolbar controls matter here:

Clock	Advances the simulation by exactly one clock cycle. This is your main tool.
Reverse	Undoes one clock cycle. Use it freely — you will want to re-examine a cycle you rushed past.
Reset	Returns the PC to the entry point and restores memory.
Run	Executes to completion without updating the display. Fast, but it records no stage information, so never use it when you need the stage table.

Step 2 — Learn to Read the Datapath

Before tracing anything, learn where to look. The table below maps each element of the Week 6 datapath onto the implementation in front of you, and names the question that element answers.

To read a value: hover over any port for a tooltip giving its name and current value; or right-click a port and choose Show value to pin a label, then right-click the label to change its radix. Clicking a wire highlights its whole route, which is the fastest way to find where a signal actually goes. A multiplexer shows a green dot on the input it has selected. A one-bit control wire is green when the signal is 1 and grey when it is 0.

#	Datapath element	Ripes component	What to read, and what it tells you
1	Program counter	`pc_reg`	out — the address of the instruction executing right now.
2	PC adder	`pc_4`	op1, op2, out — where the sequentially next instruction lives.
3	PC multiplexor	`pc_src`	select, out — whether control redirected the PC this cycle, and to where.
4	Instruction memory	`instr_mem`	addr, data_out — the 32-bit encoding being executed.
5	Decode block	`decode`	opcode, r1_reg_idx, r2_reg_idx, wr_reg_idx — which fields were pulled out of the word.
6	Immediate generator	`immediate`	imm — the immediate, assembled from its scattered fields and sign-extended.
7	Control unit	`control`	Every output. This is the Week 7 truth table, live.
8	Register file	`registerFile`	r1_out, r2_out, wr_en, data_in — the operand values and whether a result is committed.
9	ALU operand mux 1	`alu_op1_src`	select, out — register or PC? Watch this one on auipc and jal.
10	ALU operand mux 2	`alu_op2_src`	select, out — register or immediate? This is the Week 7 ALUSrc signal.
11	ALU	`alu`	op1, op2, ctrl, res — the computation actually performed.
12	Branch unit	`branch`, `br_and`, `controlflow_or`	res and the gate outputs — was the condition true, and did it change the PC?
13	Data memory	`data_mem`	addr, data_in, wr_en, data_out — used by loads and stores only.
14	Write-back multiplexor	`reg_wr_src`	select, out — which of the three candidate results is written to the register file.

Think: Element 14 chooses between three candidate results, but the Week 7 truth table has only a two-way MemtoReg signal. What is the third input, and which instructions cannot work without it?

Step 3 — Trace One Instruction, Completely

Load trace-classes.s. Clock forward until the PC reaches 0x00000008, so that add t2, t0, t1 is the instruction executing. At this point t0 = 12 and t1 = 5.

Fill in the table below from the datapath. Some entries are given so that you can confirm you are reading the right ports; the rest are yours.

Element	Port	Value
pc_reg	out	0x00000008
pc_4	out	
instr_mem	data_out	0x006283B3
decode	r1_reg_idx / r2_reg_idx	
decode	wr_reg_idx	7
control	reg_do_write_ctrl	
control	alu_op2_ctrl	
registerFile	r1_out / r2_out	12 / 5
alu_op1_src	out	
alu_op2_src	out	
alu	res	
data_mem	wr_en	
reg_wr_src	select / out	
pc_src	select / out	

Think: The data memory sits on the critical path of every instruction, yet this add never uses it. Read data_mem.addr anyway. A value is present. Where did it come from, why is it harmless, and what is it costing the machine?

Step 4 — Trace Every Instruction Class

trace-classes.s contains one instruction from each RV32I class, chosen so that each drives a different path. Step through the whole program and complete one row per class.

The source file has seventeen instruction lines, but the program viewer lists eighteen. Find the discrepancy before you go further — it matters for the Form, and it is the same pseudo-instruction question you met in Lab 2. Always count from the Instruction memory panel.

Instruction	Format	imm	op1 mux	op2 mux	alu.res	wb mux / pc mux
`addi t0, zero, 12`						
`add t2, t0, t1`						
`lui t4, 0x2B`						
`auipc t5, 0x0`						
`lw a1, 0(a0)`						
`sw t2, 4(a0)`						
`beq t0, t1, skip`						
`bne t0, t1, target`						
`jal ra, report`						
`jalr zero, ra, 0`						

As you go, answer these in your notes:

1. lui and auipc share a format but produce different results from the same immediate. Which element makes the difference, and what does each one select?
2. sw writes to memory but never writes a register. Which single control signal expresses that, and what does the write-back multiplexor do meanwhile?
3. Both branches compare the same two registers. One redirects the PC and one does not. Trace the path from the branch unit to pc_src and name every gate on the way.
4. jal and jalr both write a return address. Which input of the write-back multiplexor supplies it, and why can neither the ALU nor data memory supply it instead?

Step 5 — The Instruction Memory and Statistics Panels

Two panels in the Processor tab carry information the datapath itself does not.

- Instruction memory — columns BP, Addr, Stage and Instruction. On a single-cycle processor the Stage column marks exactly one row at a time. Watch which rows it never marks.
- Statistics — the execution-information panel: cycles elapsed, instructions retired, CPI, IPC and clock rate.

Reset, then run the program to completion. Record: the number of instructions listed in the panel; the number retired; cycles; CPI; IPC. Put these in your notes and carry the numbers to the Form.

Think: CPI here is exactly 1.00, which Week 7 called the best possible figure, and the clock rate Ripes reports is modest. Week 7 also said no processor has been built this way in decades. Using CPU time = IC x CPI x Tc, say precisely which term this design sacrificed — and by roughly what factor, given the component delays in the Week 7 worked exercise.

Step 6 — The Implementation Is Not the Textbook Diagram

The datapath in front of you is a real design, and it differs from the Week 6 figure in three substantive ways. Find all three, and for each one write a short paragraph explaining what the implementer gained. This is the most important step in Part A.

	What the Week 6 figure shows	What to look for in the implementation
A	A dedicated branch adder, separate from the ALU, computing PC + offset.	There is no second adder. Find where the branch target is actually computed, and which multiplexer makes it possible.
B	beq tested by the ALU's Zero output after a subtraction.	There is a separate comparison unit. Ask what it can do that a Zero flag cannot, and list the instructions that need it.
C	A two-input MemtoReg multiplexer.	The write-back multiplexer has three inputs. Identify the third and the instructions that depend on it.

Think: Week 6 asked why the fetch datapath uses a dedicated adder rather than the ALU. This implementation answers the opposite question — it uses the ALU where the textbook uses a dedicated adder. Both designs work. State the trade-off each one is making, and the circumstance under which you would choose each.

NUMERICAL ANSWERS — GOOGLE FORM

Submit the following via the CS3520 Lab 3 Google Form (link on Thuto). Enter numbers only. Where a hexadecimal value is asked for, give eight hex digits with no 0x prefix. One submission per student; the Form closes at the deadline.

#	Question	Answer format
Q1	trace-classes.s: how many instructions does the Instruction memory panel list?	*integer*
Q2	trace-classes.s: how many of those are never executed?	*integer*
Q3	Single-cycle processor: the CPI reported in the Statistics panel.	*2 d.p.*
Q4	Value of t4 after lui t4, 0x2B, in decimal.	*integer*
Q5	Value of t5 after auipc t5, 0x0, in decimal.	*integer*
Q6	The 32-bit encoding of add t2, t0, t1.	*8 hex digits*
Q7	The value on the immediate generator output while sw t2, 4(a0) executes.	*integer*
Q8	The value on alu.res while sw t2, 4(a0) executes, in decimal.	*integer*
Q9	The value on the branch unit output while beq t0, t1, skip executes.	*0 or 1*
Q10	The value on pc_src.select while bne t0, t1, target executes.	*0 or 1*

Every one of these can be read directly off the simulator or derived from your trace tables. If you find yourself guessing, you have skipped a step — go back and find the port.

WHAT TO SUBMIT

Two submissions. Both are required.

1. The Google Form, with your twenty numerical answers.
2. On Thuto, the URL of your public GitHub repository. Before you submit it, check that labs/2026/lab-3/ contains everything below.
   - trace-tables.md — your completed tables from Steps 3 and 4
   - datapath-notes.md — your answers to Step 4's four questions and all three findings from Step 6, with the reasoning
   - A commit history that shows the work progressing, with messages that say what changed

> *The predictions carry marks whether or not they turned out to be right. A wrong prediction you can explain is \| evidence of understanding; a right answer with no reasoning behind it is not.*

GETTING HELP

- Lab demonstrators are on hand throughout the session — ask before searching online.
- Outside lab hours: post on the CS3520 course forum, or email ko.setetemela@nul.ls.
- Consultation hours are posted on the office notice board.